

Intelligent Sudoku Solver

1.Objective

The goal of this work is to model standard 9x9 Sudoku strictly as a Constraint Satisfaction Problem (CSP) and solve it without naive brute force. The implementation uses AC-3 for domain reduction, recursive Backtracking Search, and the Minimum Remaining Values (MRV) heuristic for variable selection. Input and output follow the required 81-character line format.

2. CSP Formulation

- Variables: 81 variables $X(r,c)$, one for each cell of the 9x9 grid.
- Domains: a given cell has a singleton domain {value}; an empty cell initially has domain $\{1,...,9\}$.
- Constraints: every pair of cells sharing a row, column, or 3x3 box must satisfy $X_i \neq X_j$.
- Goal state: all 81 domains are singleton and every row, column, and 3x3 box is consistent.

3. Object-Oriented Architecture

Component	Responsibility	Important Methods
Sudokus	Parsing, variables, domains, peers and Sudoku constraints	_validate_puzzle, _build_domains, checkAxes, checkIfUsed
SudokuSolver	AC-3 propagation and recursive search	revise, ac3, select_unassigned_variable, backtrack, solve
SolveStats	Collects measurable execution statistics	elapsed_seconds, node_expansions, backtracks, ac3_revisions

4. AC-3 Preprocessing and Constraint Propagation

Initial preprocessing: The AC-3 queue starts with all directed arcs (X_i, X_j) between peer cells. REVISE removes a value x from $D(X_i)$ when no value y in $D(X_j)$ supports the not-equal constraint $x \neq y$. If a domain becomes empty, the puzzle or tentative branch is inconsistent.

Incremental reuse: After assigning a tentative value during backtracking, the same AC-3 engine is run only on arcs affected by that assignment. This preserves the required full AC-3 preprocessing while also pruning future search branches early.

AC-3 complexity: For a binary CSP, the standard upper bound is $O(e d^3)$, where e is the number of arcs and d is the maximum domain size. In Sudoku, $d \leq 9$, so aggressive domain reduction is practical.

5. Backtracking Search and Heuristics

- MRV enabled: select the unassigned cell with the smallest current domain.
- MRV disabled: select the first unassigned cell in row-major order.
- Value ordering: both modes use the same least-constraining-value (LCV) ordering, so the ablation changes only variable selection.
- Each attempted value increments `node_expansions`. A rejected or failed branch increments backtracks.
- After each tentative assignment, localized AC-3 performs constraint propagation before the next recursive call.

6. Input and Output

Input: `puzzles.txt` contains one 81-character puzzle per line. Digits 1-9 are givens and 0 represents an empty cell.

Output: `solutions.txt` contains one solved 81-character line for each input puzzle, with no 0 digits.

7. Ablation Study: MRV Enabled vs Disabled

Test puzzle: The third provided puzzle was treated as the Hard puzzle. Each timing value below is the average of 5 runs of the same implementation. AC-3 behavior and value ordering are identical in both modes; only variable selection changes.

Configuration	Average time (s)	Node expansions	Backtracks
Backtracks	0.378388	1828	1815
MRV disabled (sequential)	0.699072	3886	3870

Time reduction = $(0.699072 - 0.378388) / 0.699072 \times 100 = 45.87\%$

Node expansion reduction = $(3886 - 1828) / 3886 \times 100 = 52.96\%$

Backtrack reduction = $(3870 - 1815) / 3870 \times 100 = 53.10\%$

Observed speedup = $0.699072 / 0.378388 = 1.85x$

Interpretation: On this puzzle, MRV reduced node expansions by 52.96% and backtracks by 53.10%. The measured runtime decreased by 45.87%, corresponding to about 1.85x speedup. This supports the idea that choosing the most constrained variable can shrink the effective search tree. Exact execution time can vary slightly across computers, but node and backtrack counts are deterministic for this implementation and candidate ordering.

8. Correctness Checks

- Every input line is validated to contain exactly 81 digits.
- AC-3 detects an inconsistency if any domain becomes empty.
- A complete board is accepted only when all domains are singleton and no peer pair contains the same value.
- The generated solutions contain exactly 81 digits and no zeros.

9. Conclusion

The solver represents Sudoku as a binary CSP, performs AC-3 domain pruning, and completes the search through recursive backtracking. MRV improves the measured Hard-puzzle search by reducing the number of explored candidate assignments and failed branches. The program also follows the required text-file input/output format and keeps the CSP, search logic, and statistics separated through an object-oriented design.